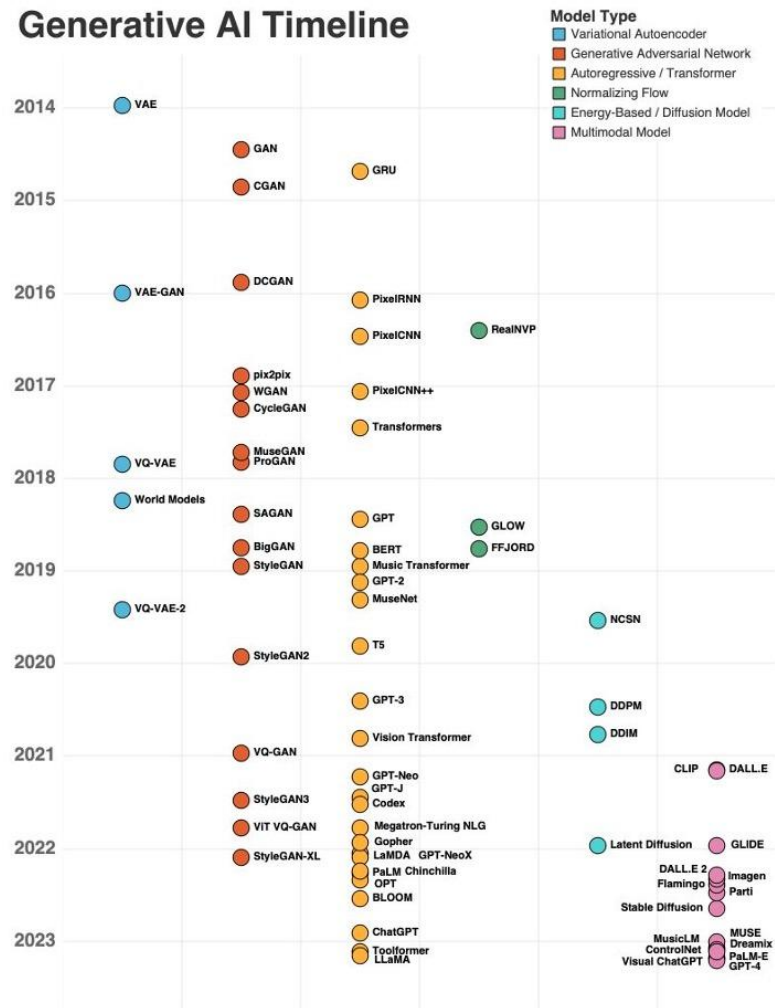


Generative model

Mark Blumenau

What is it about?

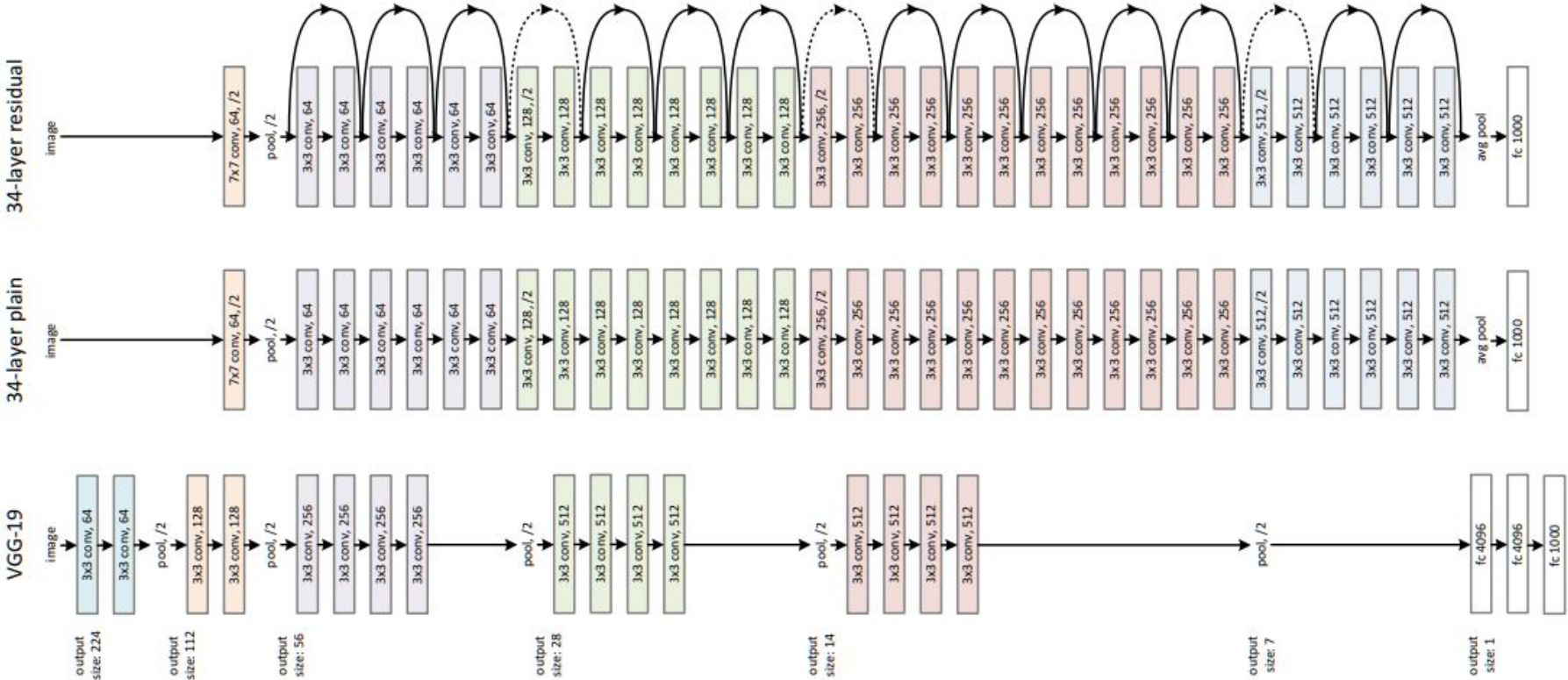
Generative AI Timeline



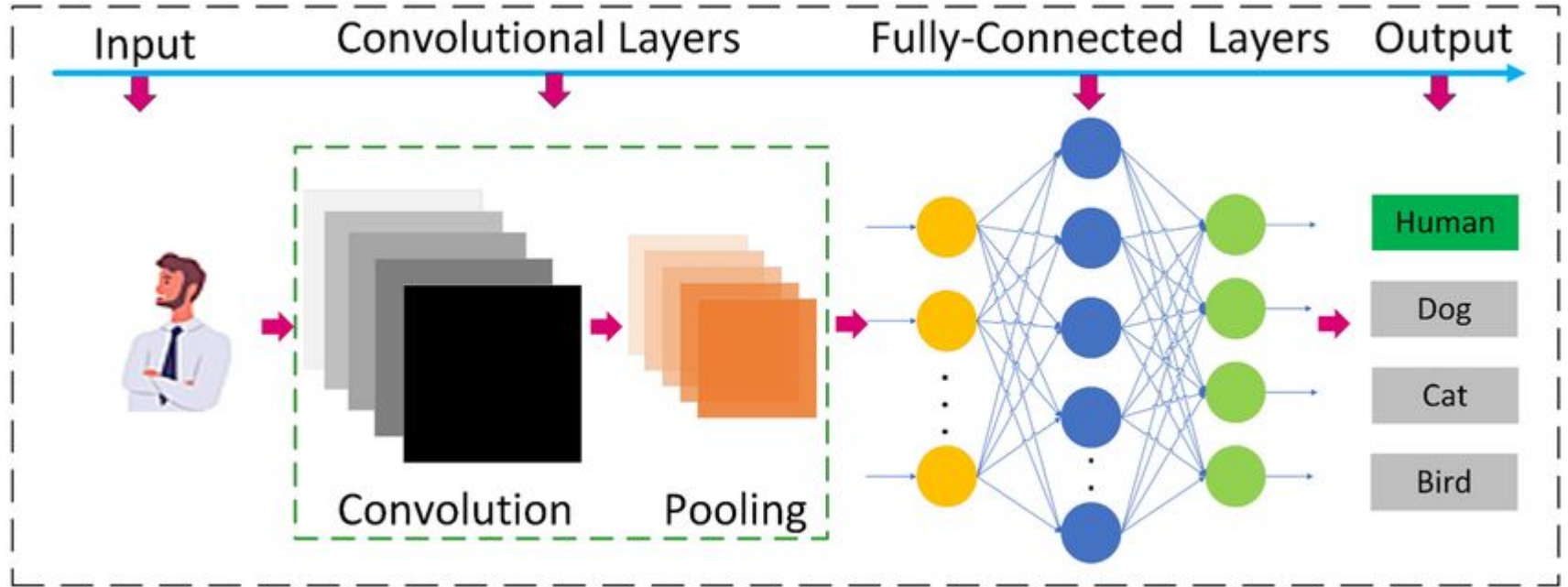


Credit: Elena
Kantonistova

What we know



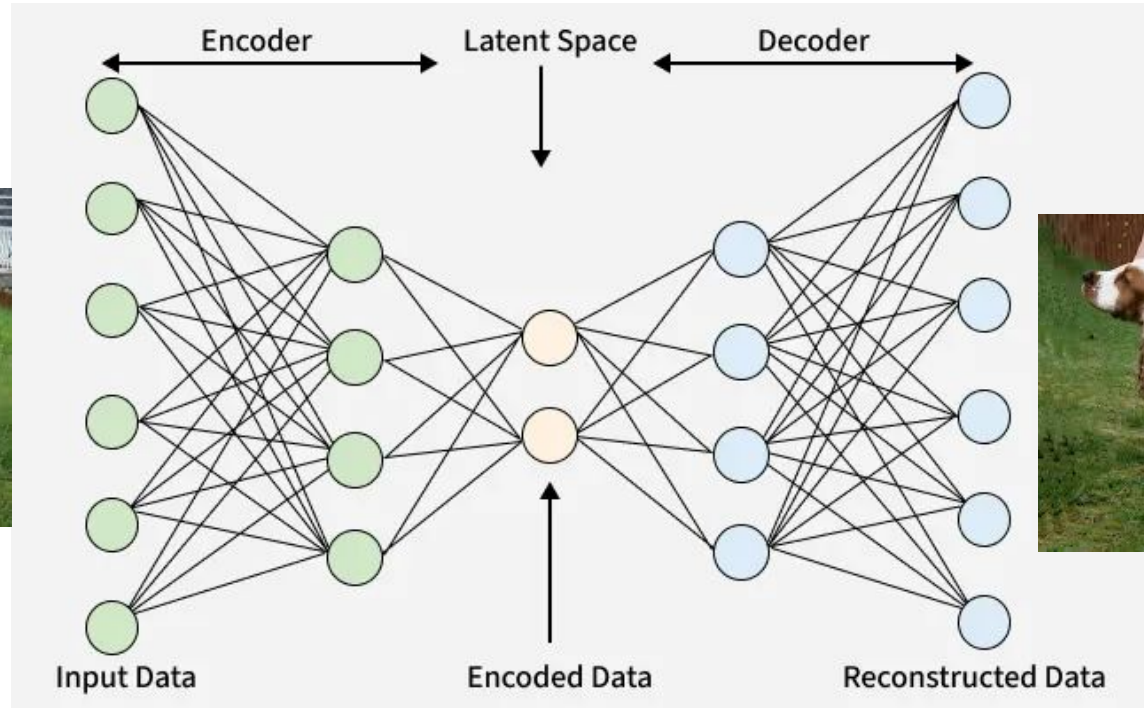
Supervised embeddings



What if we don't have any labels?

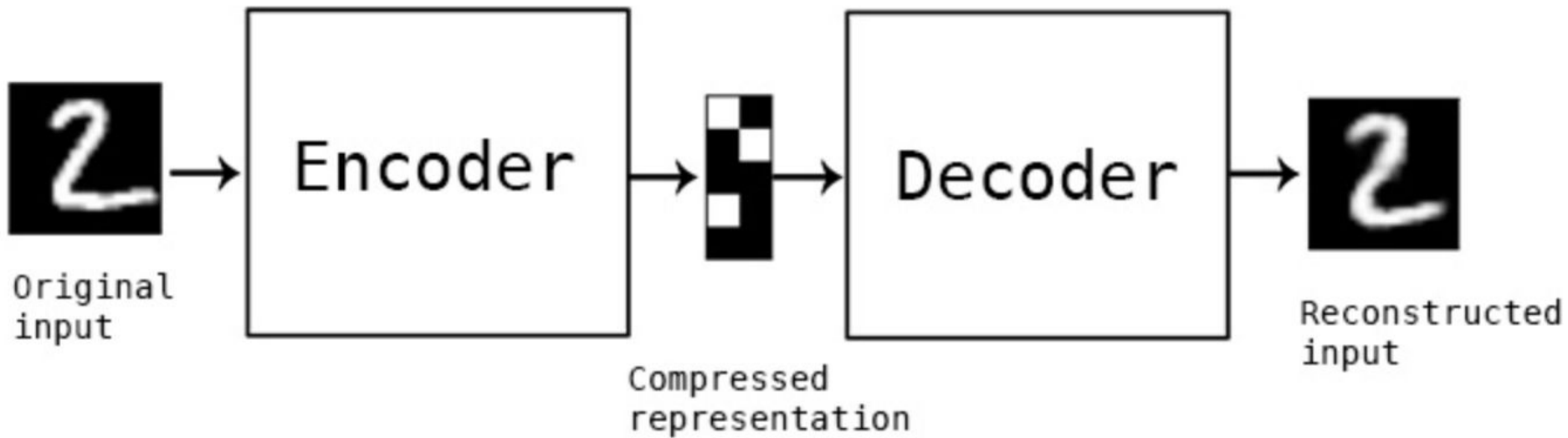
Do we watch some older lectures?

No!

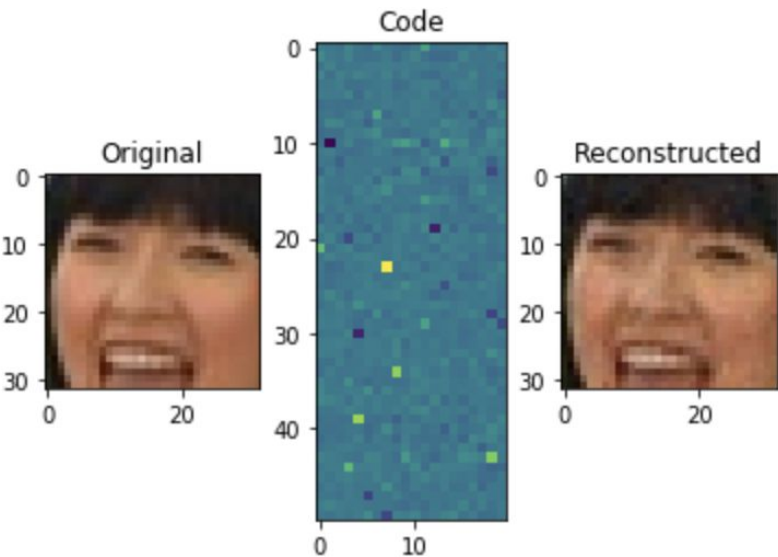
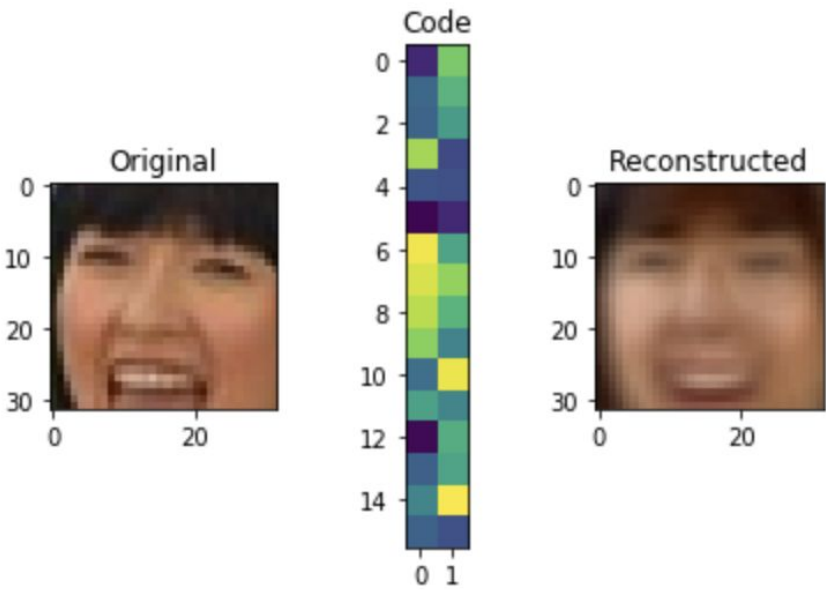


How to train this model?

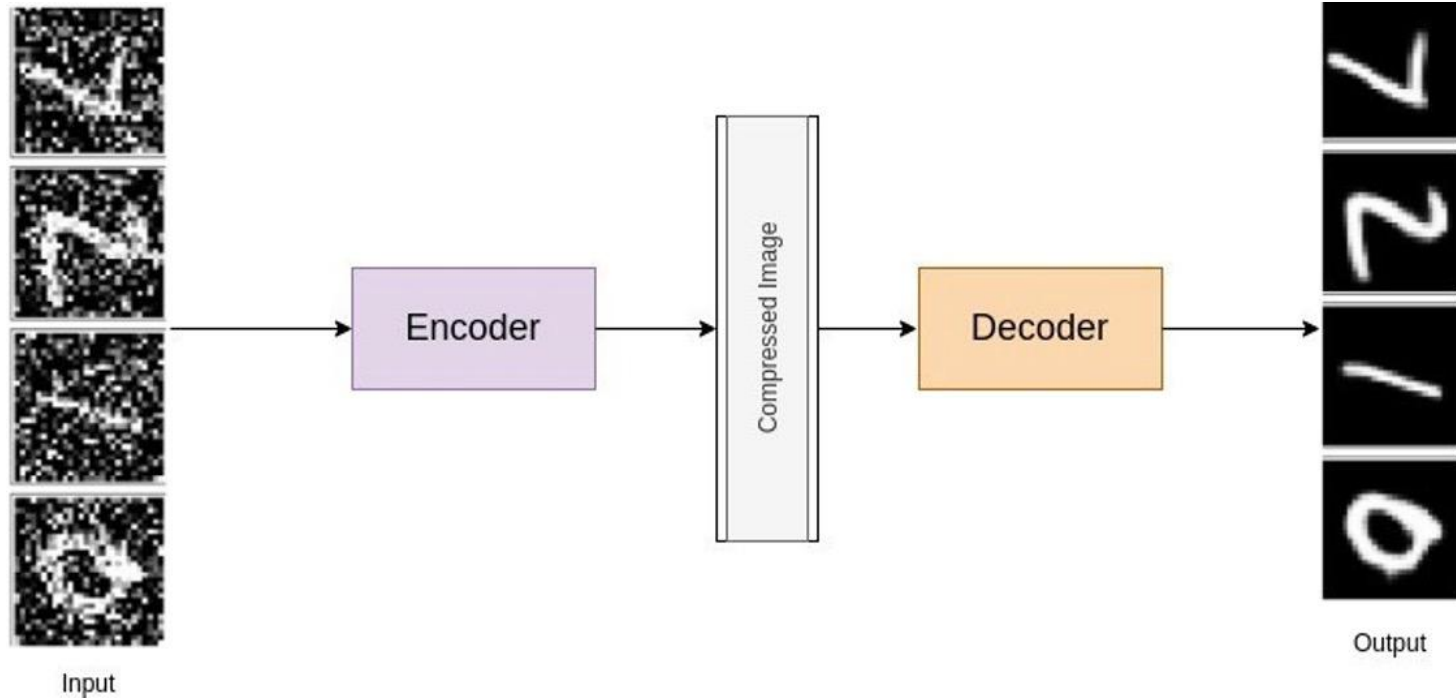
$$L\left(x_i, g\left(f\left(x_i\right)\right)\right) \rightarrow \min$$



More numbers = better reconstruction!

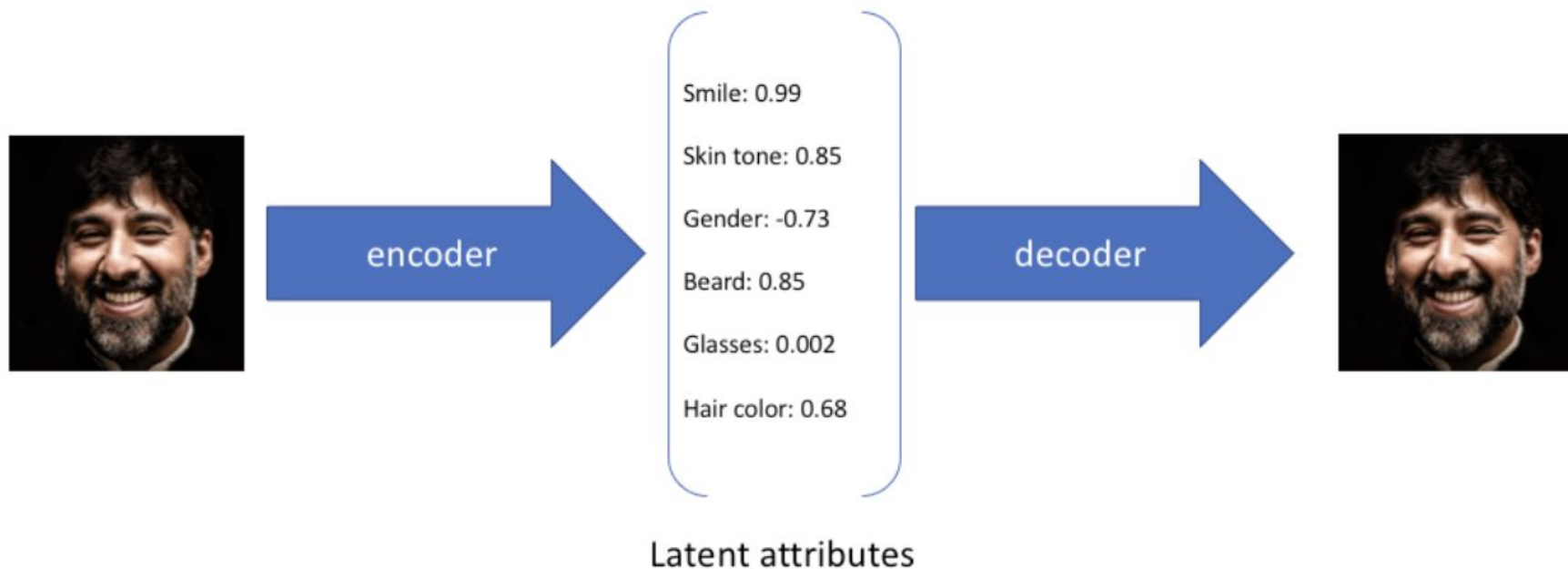


We can even denoise the image!

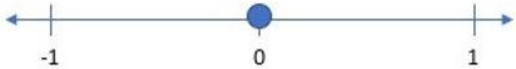
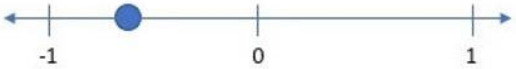


Let's discuss why we need this model...

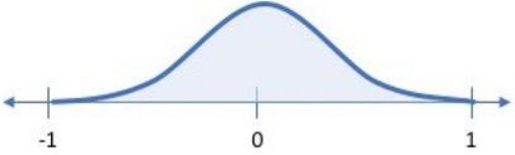
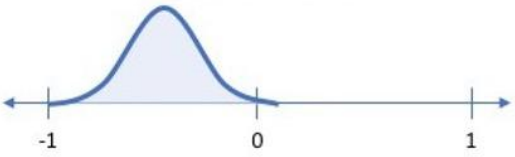
Variational AE = AE but better



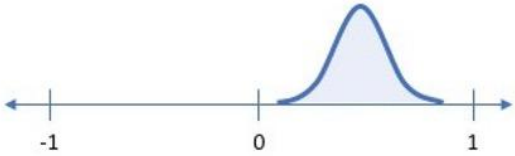
Smile (discrete value)

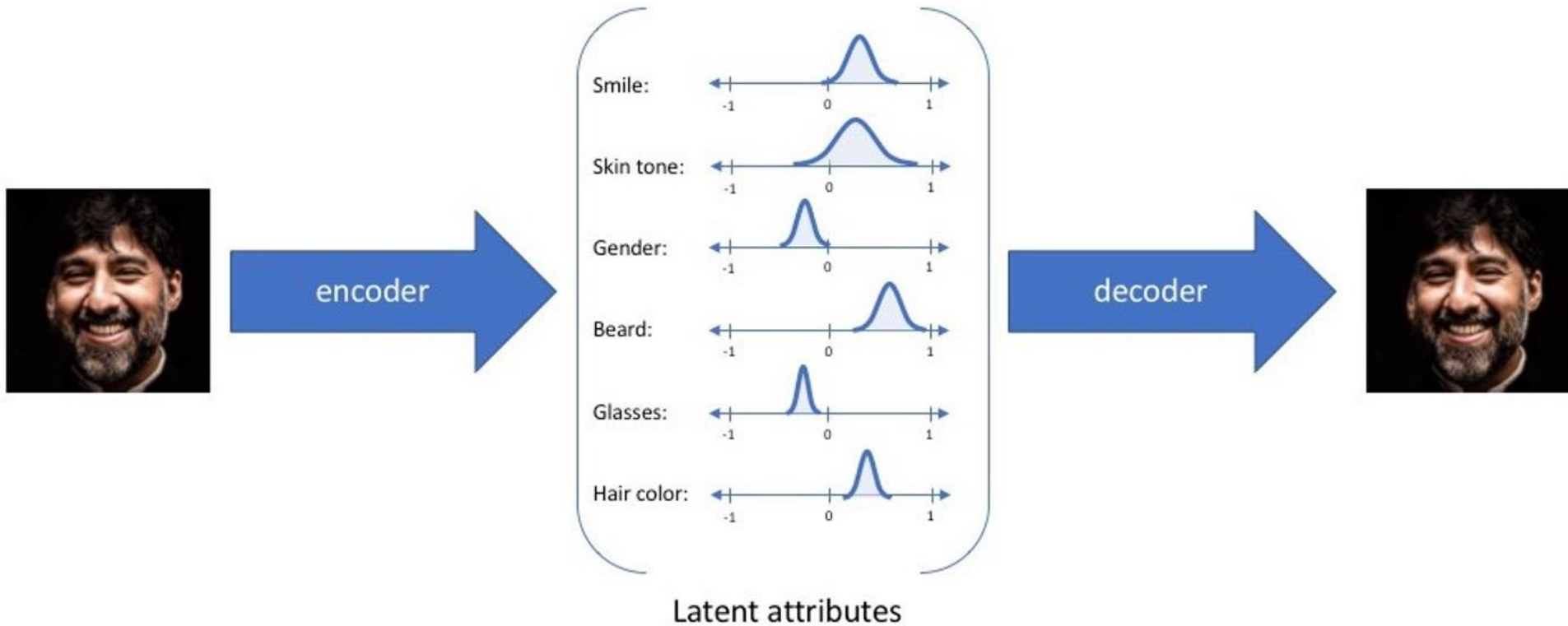


Smile (probability distribution)



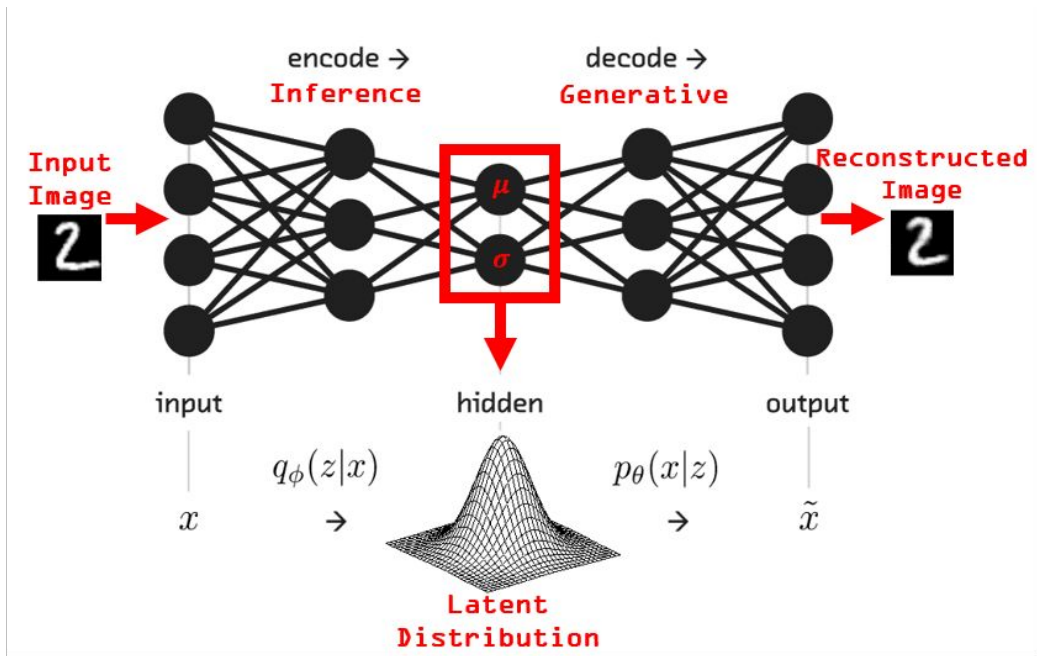
VS.



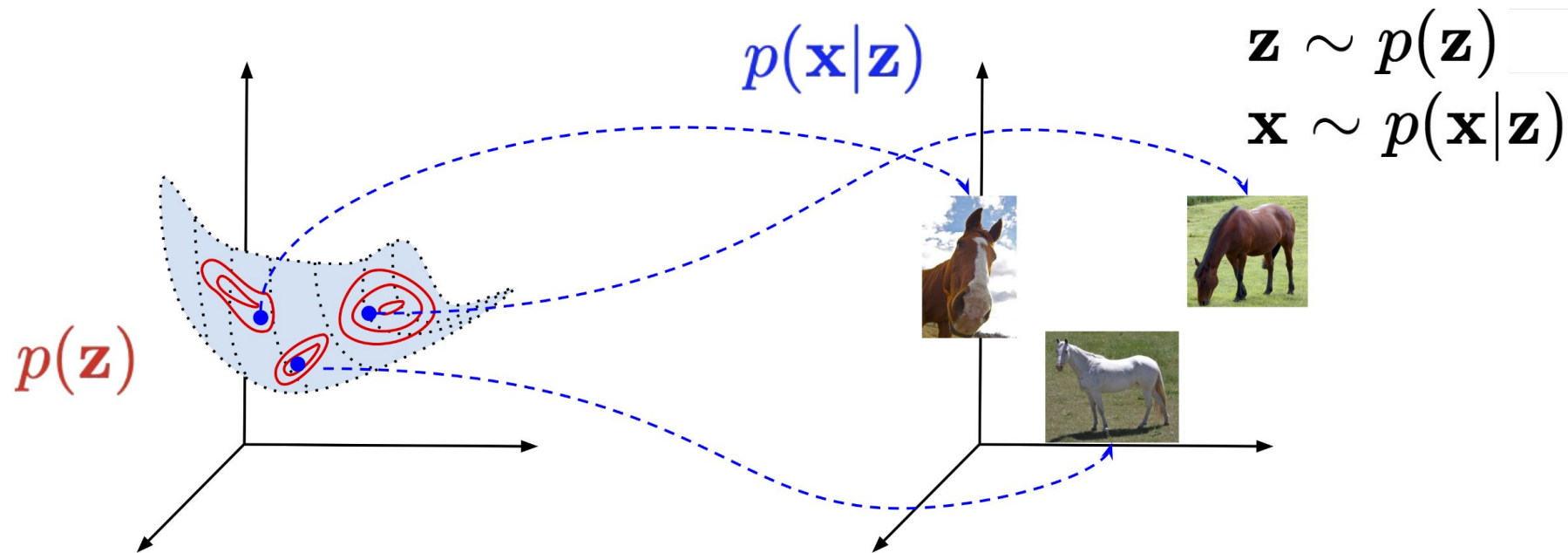


Basically

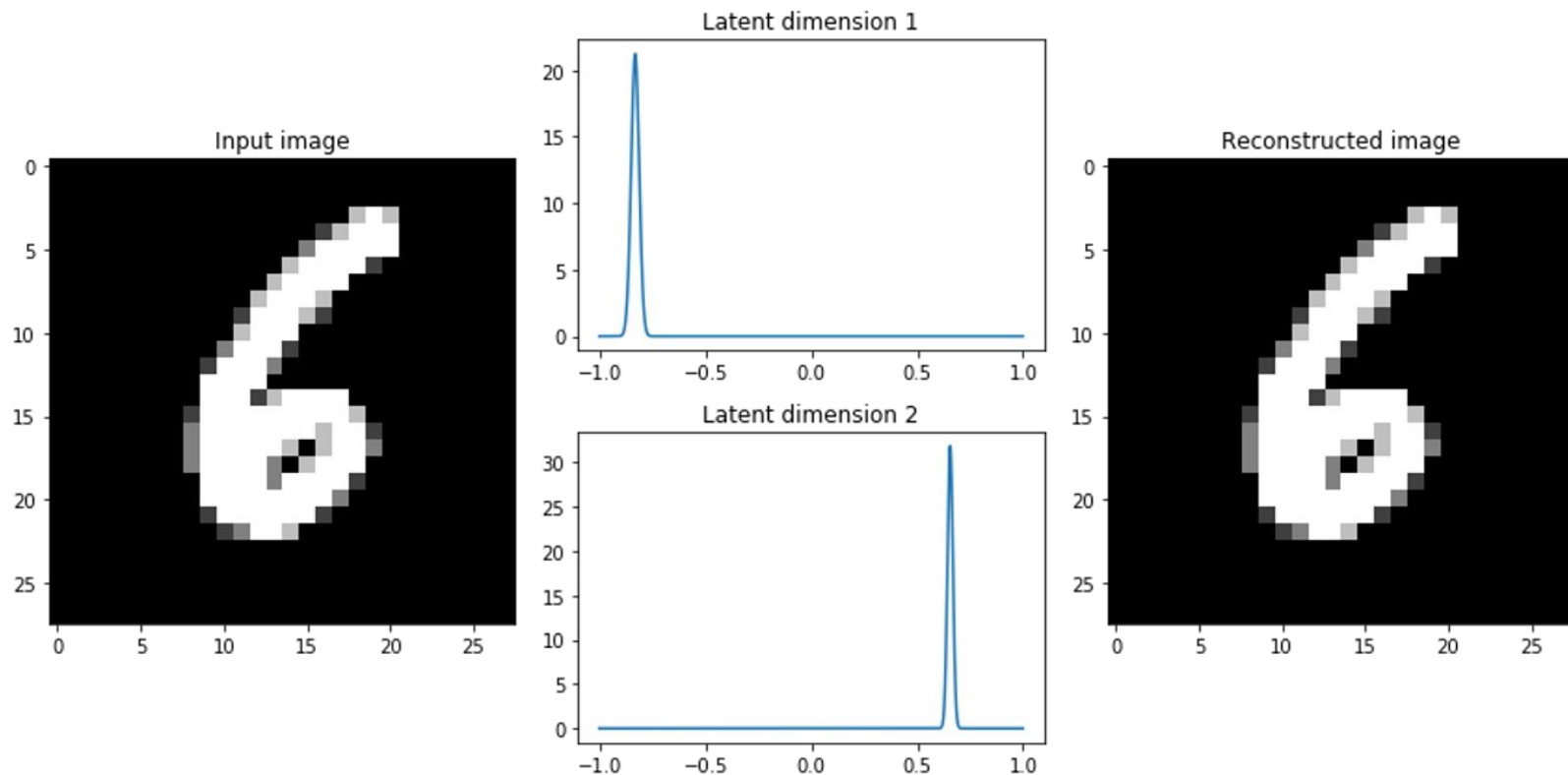
$$\text{encoder}(x) = (\mu(x), \sigma(x))$$



Another way of thinking



What we don't want



How do we train this model?

$$E_{q(z|x)} \log p(x|z) - KL(q(z|x) || p(z)) \rightarrow \max$$

Reconstruction

Regularization

$$D_{\text{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}.$$

Reconstruction is often MSE

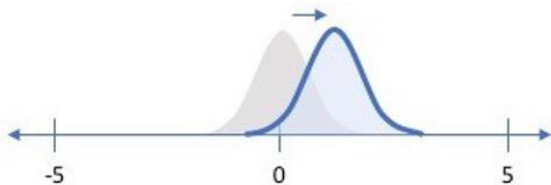
Assuming decoder outputs the mean of a Gaussian

$$p(x \mid z) = \mathcal{N}(x \mid \mu_{\theta}(z), \sigma^2 I)$$

$$\log p(x \mid z) = -\frac{1}{2\sigma^2} \|x - \mu_{\theta}(z)\|^2 - \frac{d}{2} \log(2\pi\sigma^2)$$

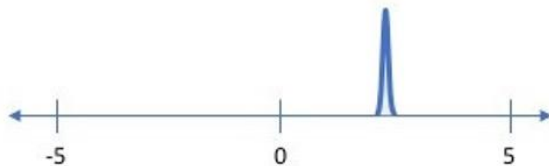
$$-\mathbb{E}_{q(z|x)}[\log p(x|z)] = \frac{1}{2\sigma^2} \mathbb{E}_{q(z|x)} \|x - \mu_{\theta}(z)\|^2 + \text{const}$$

Penalizing reconstruction loss encourages the distribution to describe the input



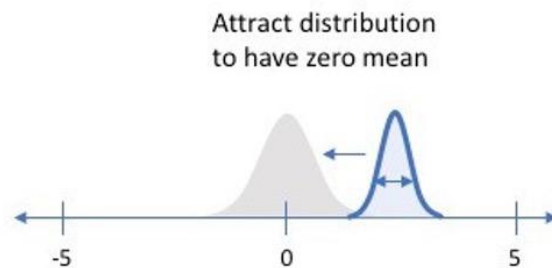
Our distribution deviates from the prior to describe some characteristic of the data

Without regularization, our network can “cheat” by learning narrow distributions



With a small enough variance, this distribution is effectively only representing a single value

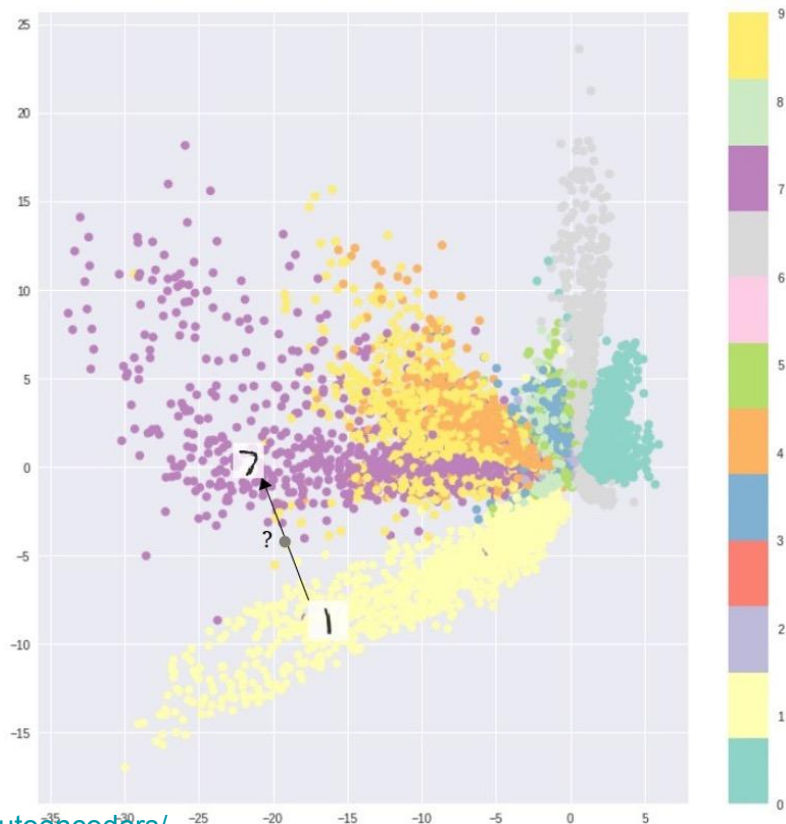
Penalizing KL divergence acts as a regularizing force



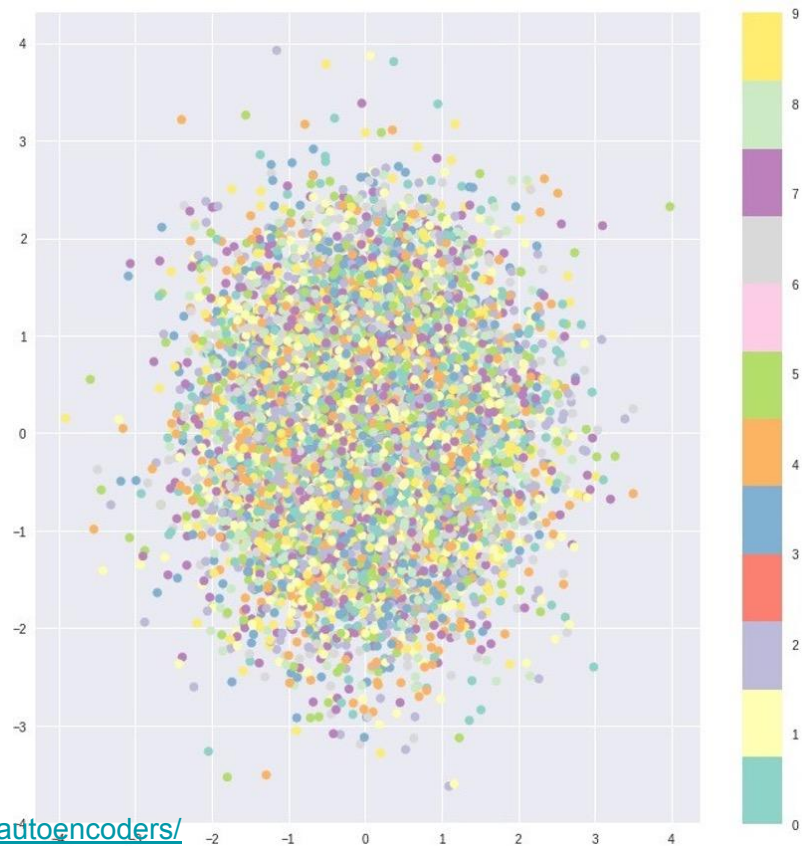
Attract distribution to have zero mean

Ensure sufficient variance to yield a smooth latent space

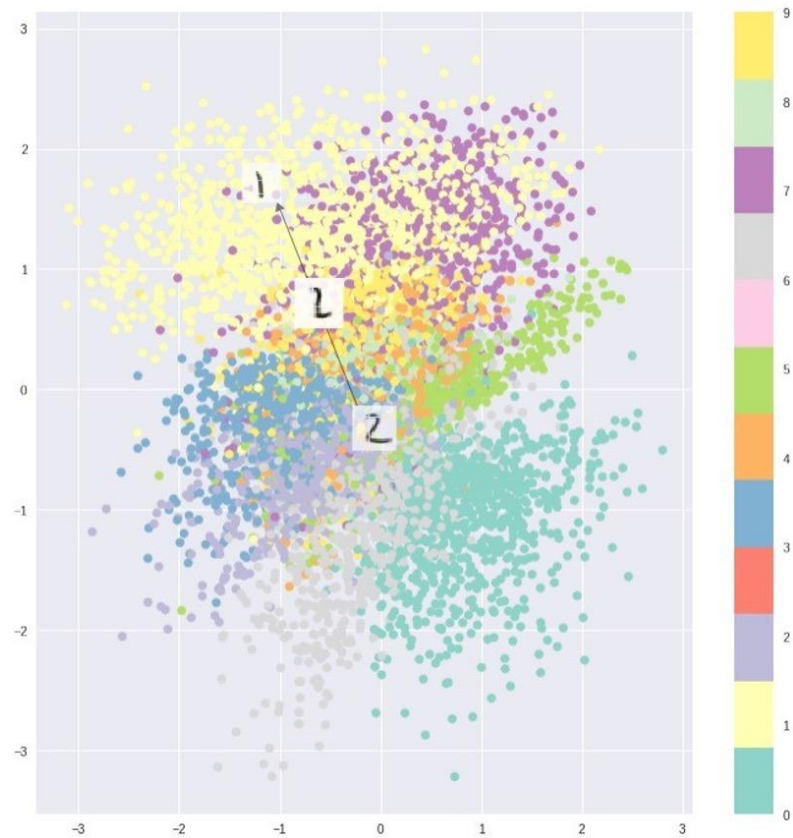
Only reconstruction

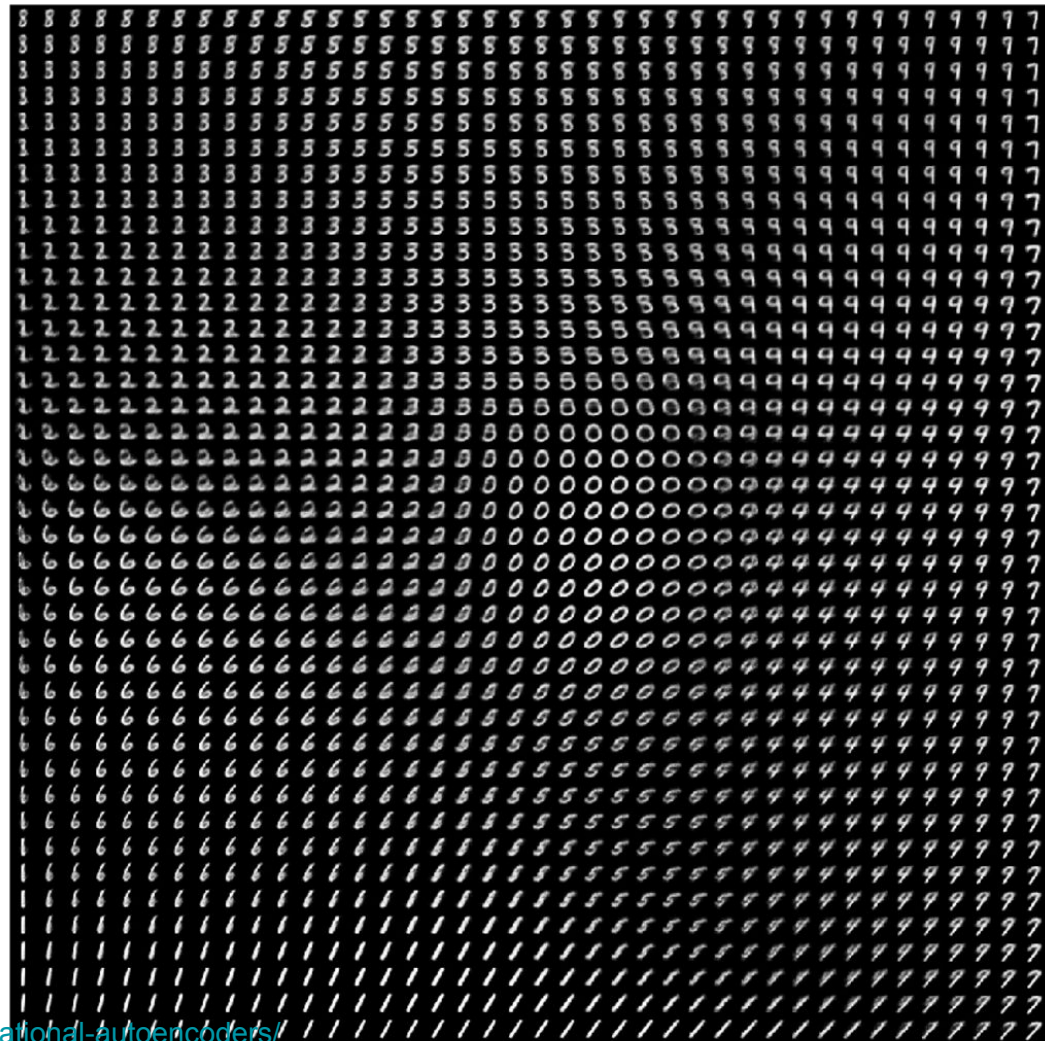


Only KL

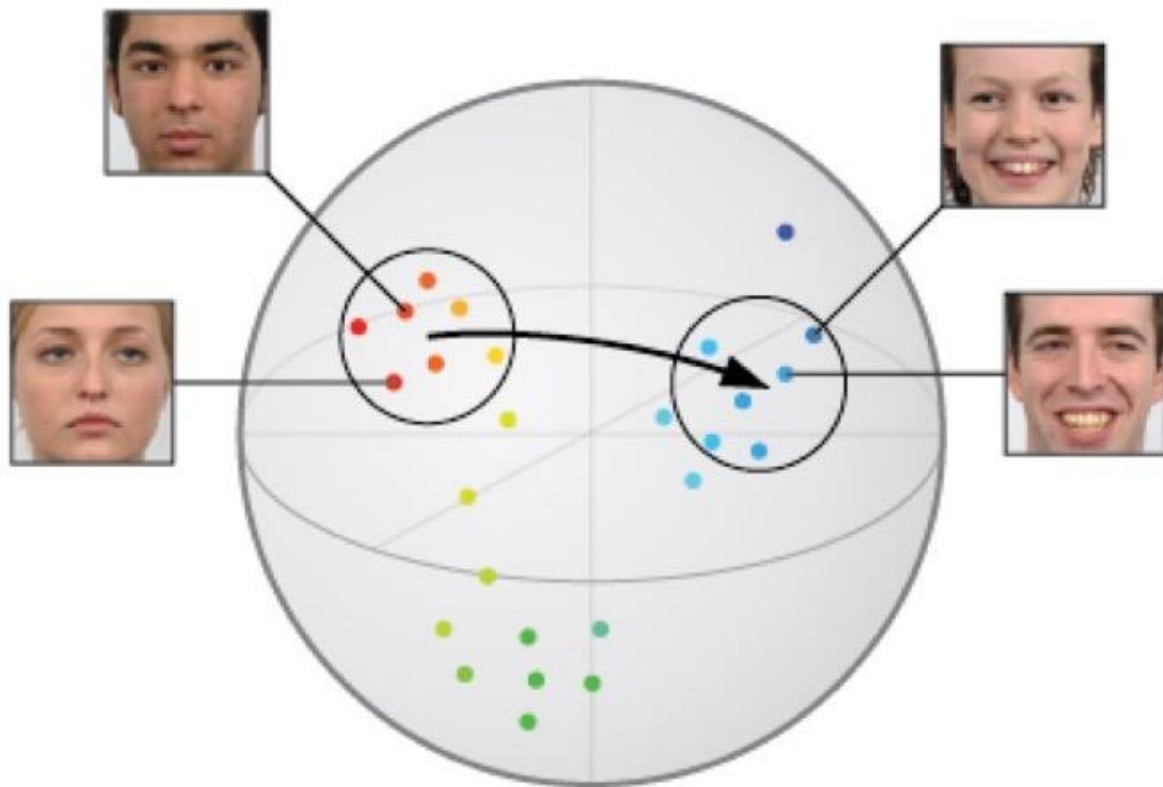


Both





Smile!



Reconstruction

Gender-biased
smilevector

Gender-balanced
smilevector

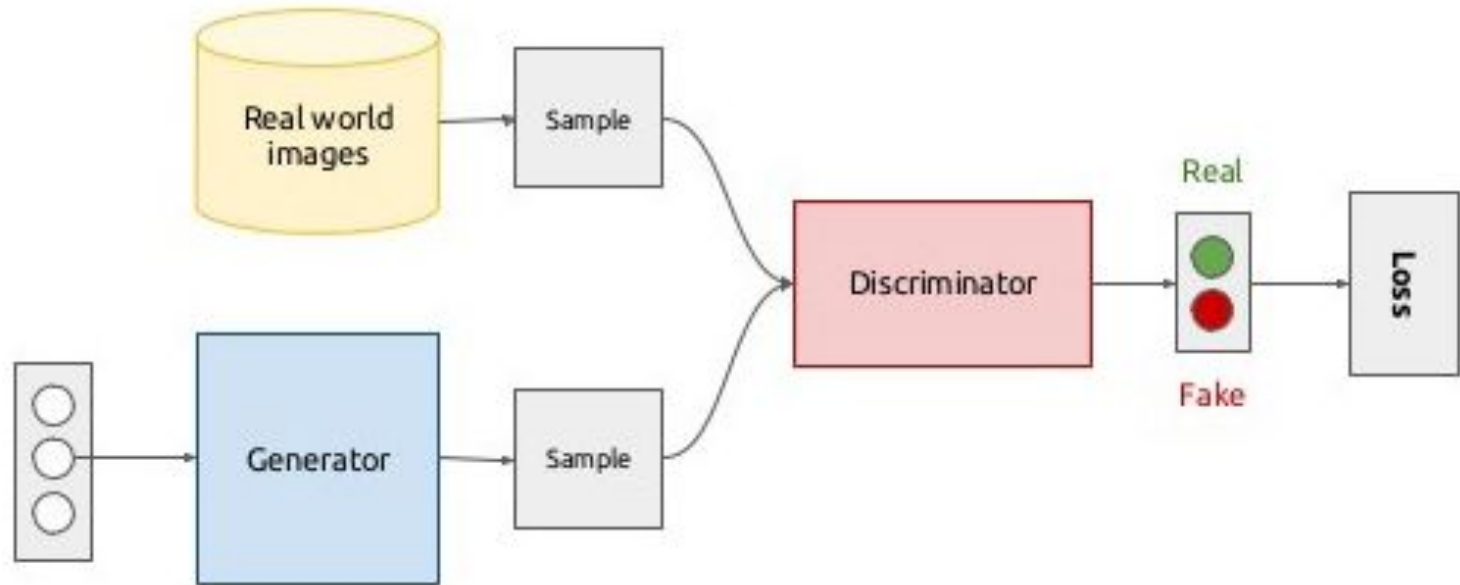


Generation

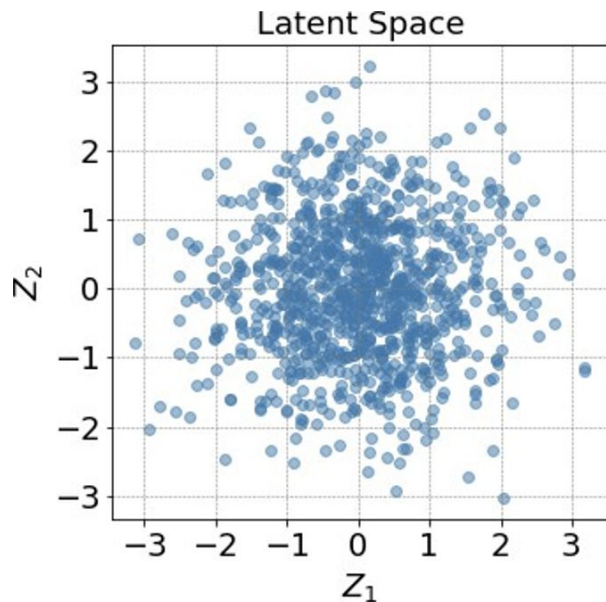


<https://gaussian37.github.io/deep-learning-chollet-8-4/>

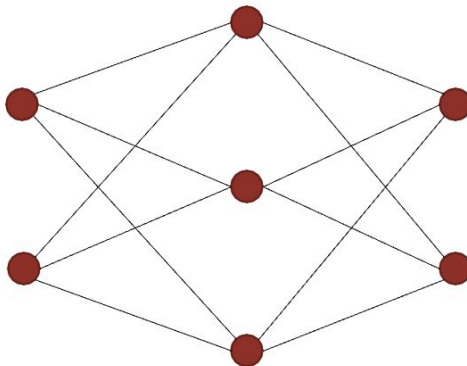
New model! GAN



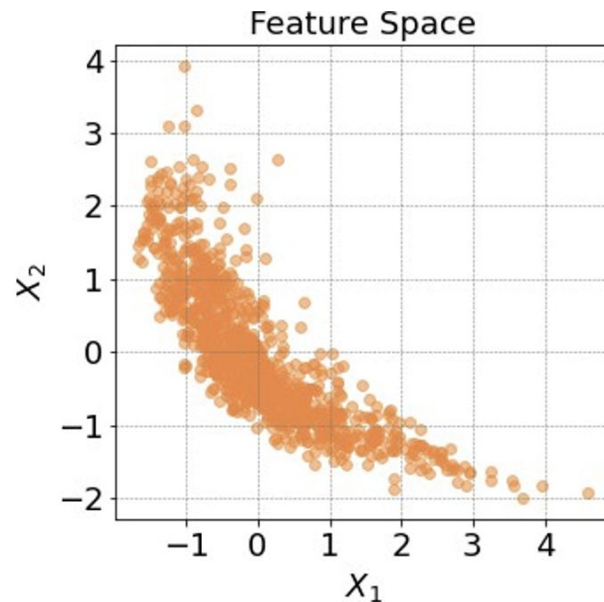
Generator



Z

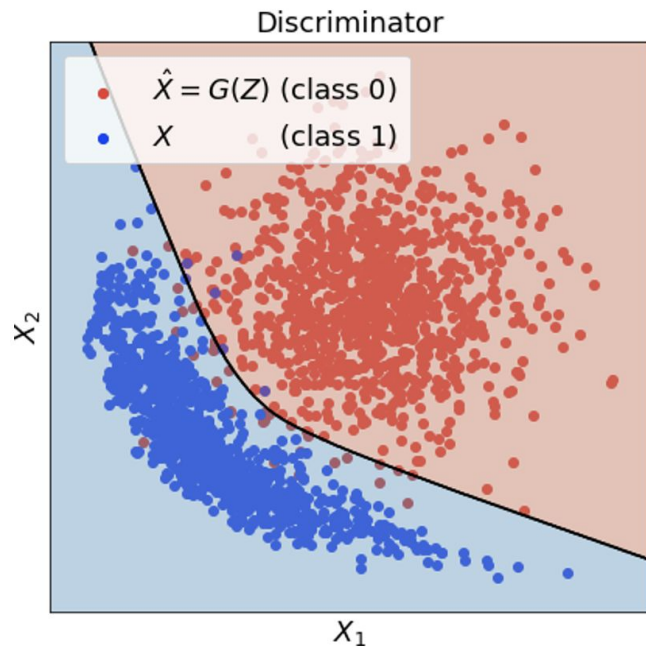


$G(Z)$



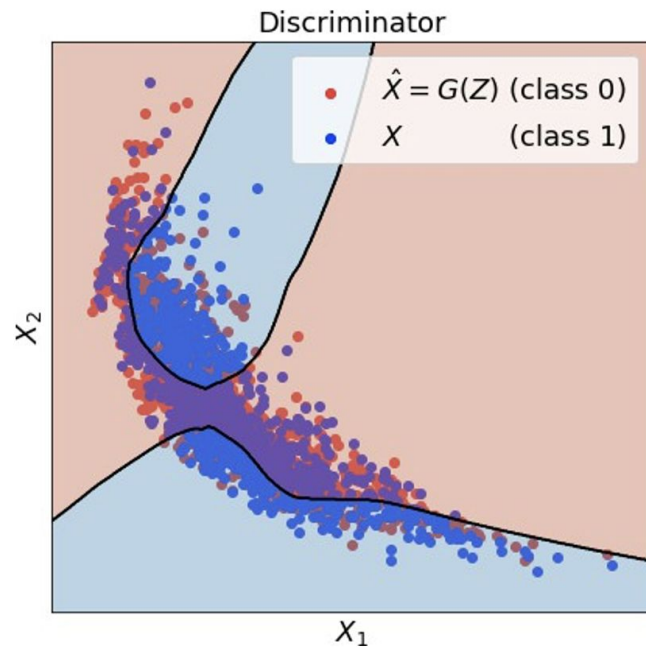
$X = G(Z)$

Discriminator



ROC AUC = 0.995

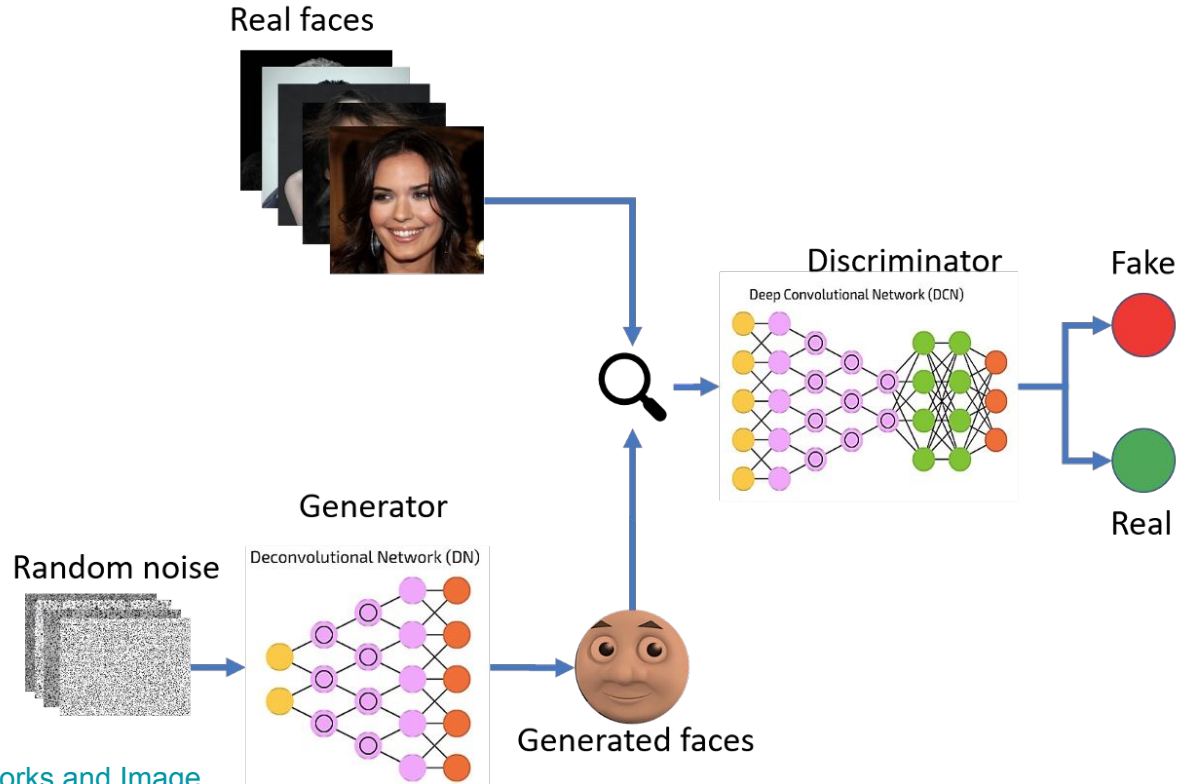
Log Loss = 0.135



ROC AUC = 0.554

Log Loss = 0.689

In summary



How to train?

$$L(\mathbf{D}, \mathbf{G}) = -\frac{1}{n} \sum_{x_i \in X} \log \mathbf{D}(x_i) - \frac{1}{n} \sum_{z_i \in Z} \log(1 - \mathbf{D}(\mathbf{G}(z_i)))$$

$$\max_{\mathbf{G}} \min_{\mathbf{D}} L(\mathbf{D}, \mathbf{G})$$

Results



2014



2015



2016

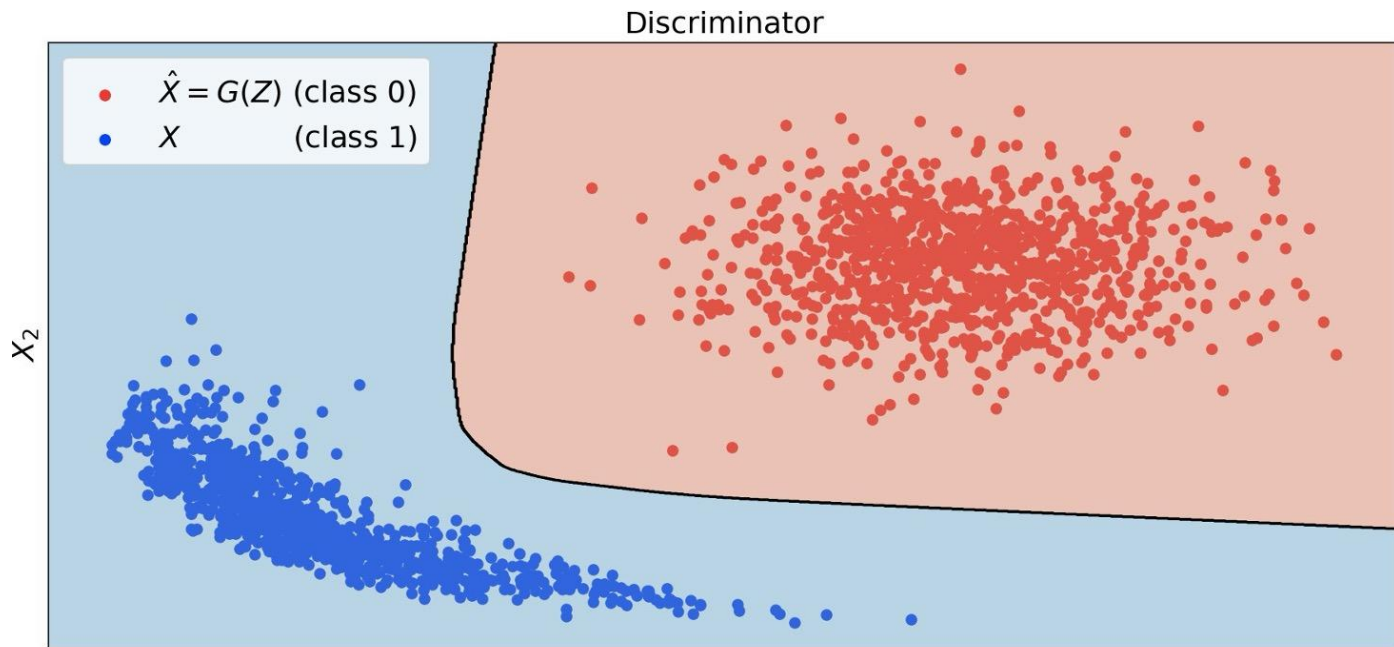


2017



2018

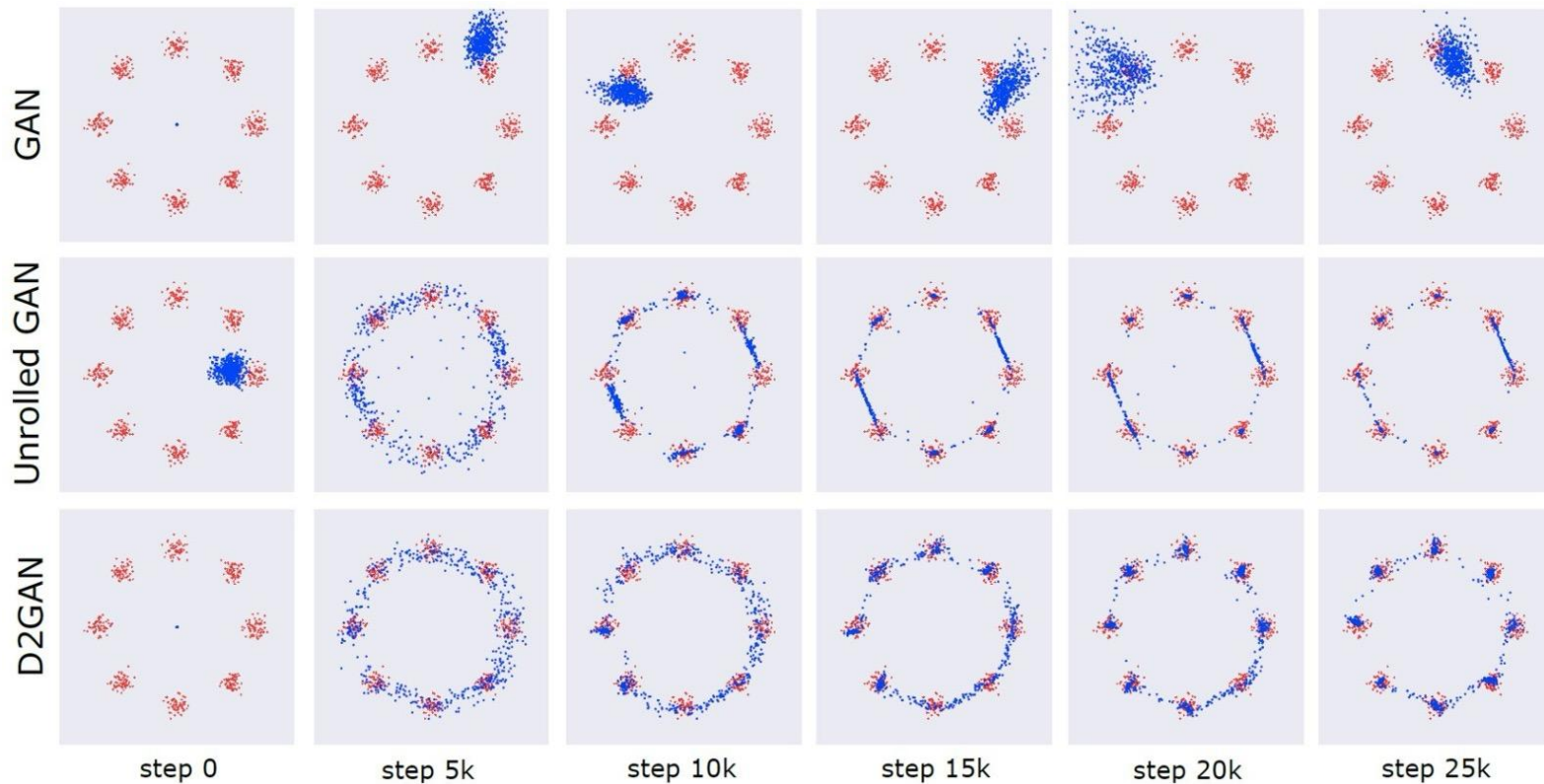
Problems



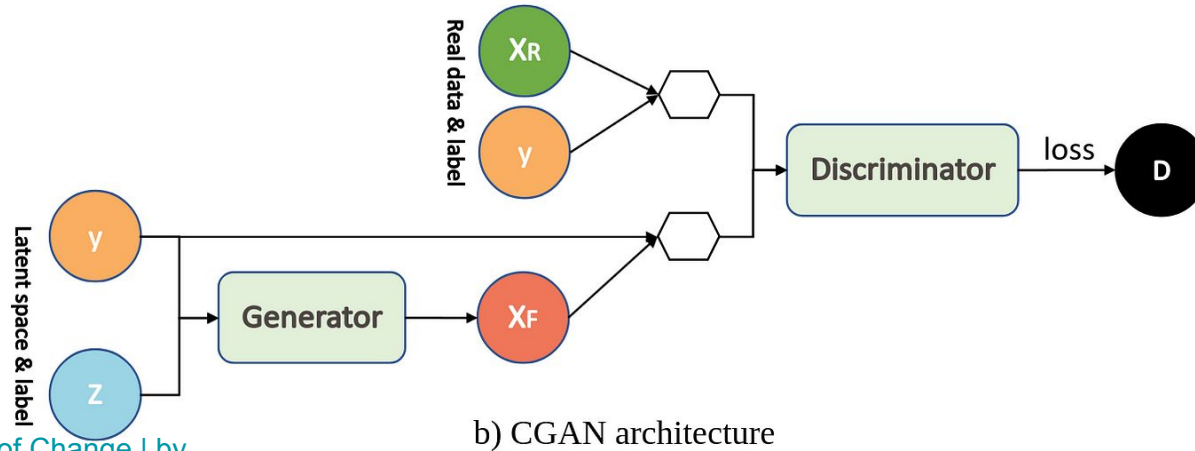
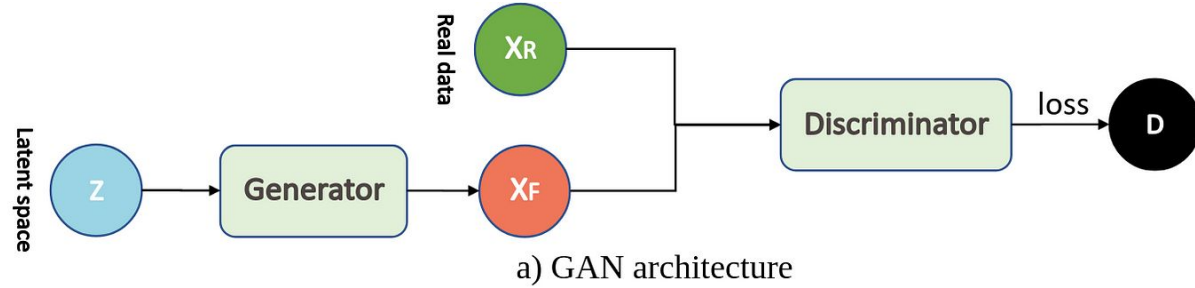
$$L(\mathbf{D}, \mathbf{G}) = -\frac{1}{n} \sum_{x_i \in X} \log \mathbf{D}(x_i) - \frac{1}{n} \sum_{z_i \in Z} \log(1 - \mathbf{D}(\mathbf{G}(z_i)))$$

Credit: Elena Kantonistova

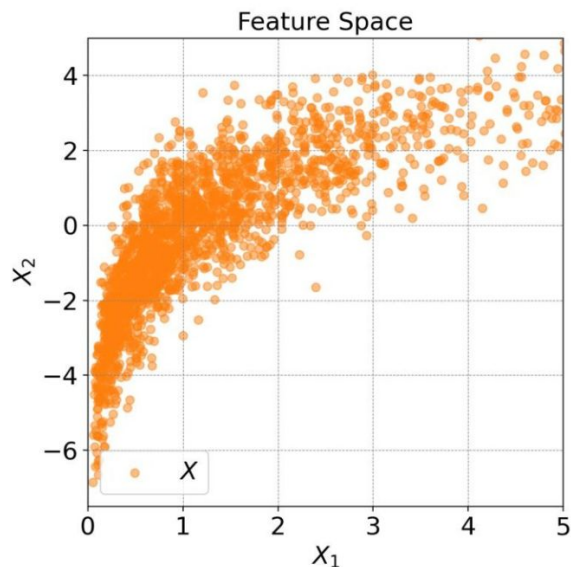
Mode collapse



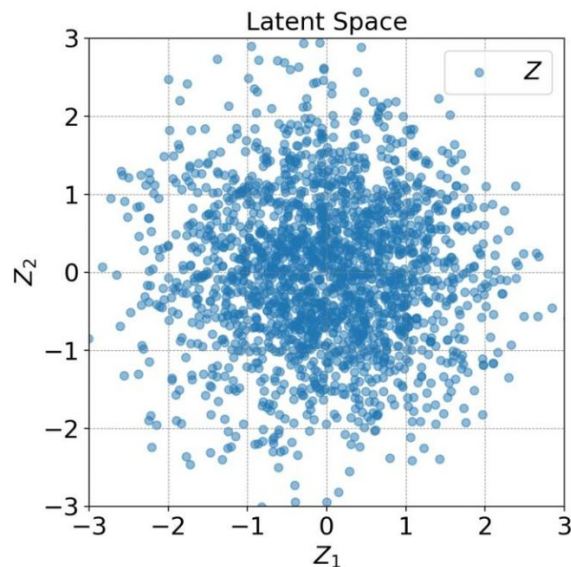
Conditional GAN



New model – Normalizing Flows!

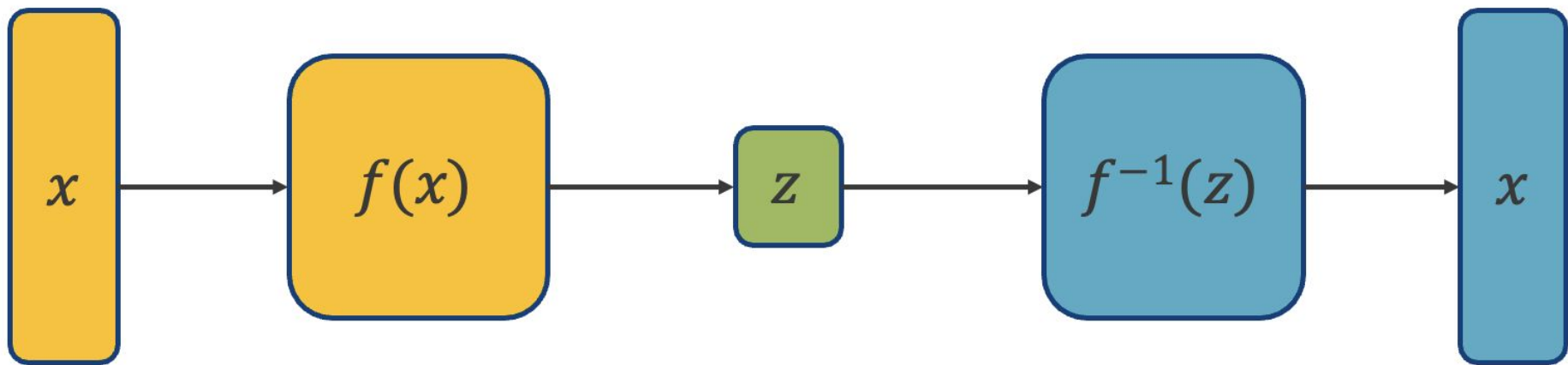


$$z = f(x) - ?$$



$$p_z(z) - \text{known}$$

Model



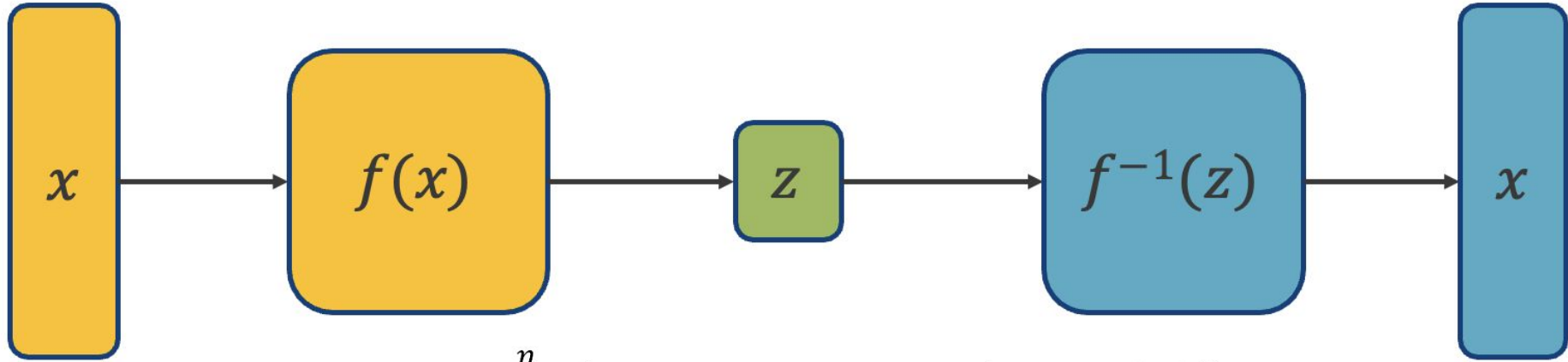
How to train?

$$L = -\frac{1}{n} \sum_{i=1}^n \log p_x(x_i)$$

$$p_x(x_i) = p_z(f(x_i)) \left| \det \frac{\partial f(x_i)}{\partial x_i} \right|$$

$$L = -\frac{1}{n} \sum_{i=1}^n \left(\log p_z(f(x_i)) + \log \left| \det \frac{\partial f(x_i)}{\partial x_i} \right| \right)$$

Model



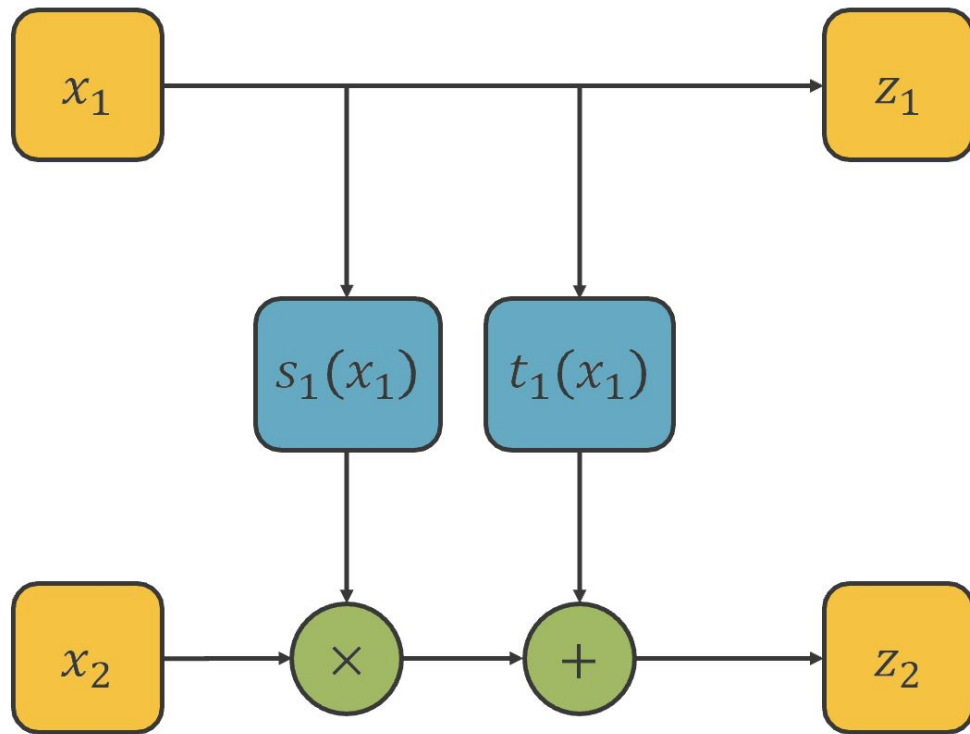
$$L = -\frac{1}{n} \sum_{i=1}^n \left(\log p_z(f(x_i)) + \log \left| \det \frac{\partial f(x_i)}{\partial x_i} \right| \right)$$

However!

The function must be:

- 1) Differentiable
- 2) Invertible

Real NVP



Real NVP

s, t – neural networks

$$z = \textcolor{brown}{f}(x) = \begin{cases} z_{1:d} = x_{1:d} \\ z_{d+1:D} = x_{d+1:D} \odot \exp(\textcolor{blue}{s}(x_{1:d})) + \textcolor{blue}{t}(x_{1:d}) \end{cases}$$

$$x = \textcolor{brown}{f}^{-1}(z) = \begin{cases} x_{1:d} = z_{1:d} \\ x_{d+1:D} = (z_{d+1:D} - \textcolor{blue}{t}(x_{1:d})) \odot \exp(-\textcolor{blue}{s}(x_{1:d})) \end{cases}$$

Math of Real NVP

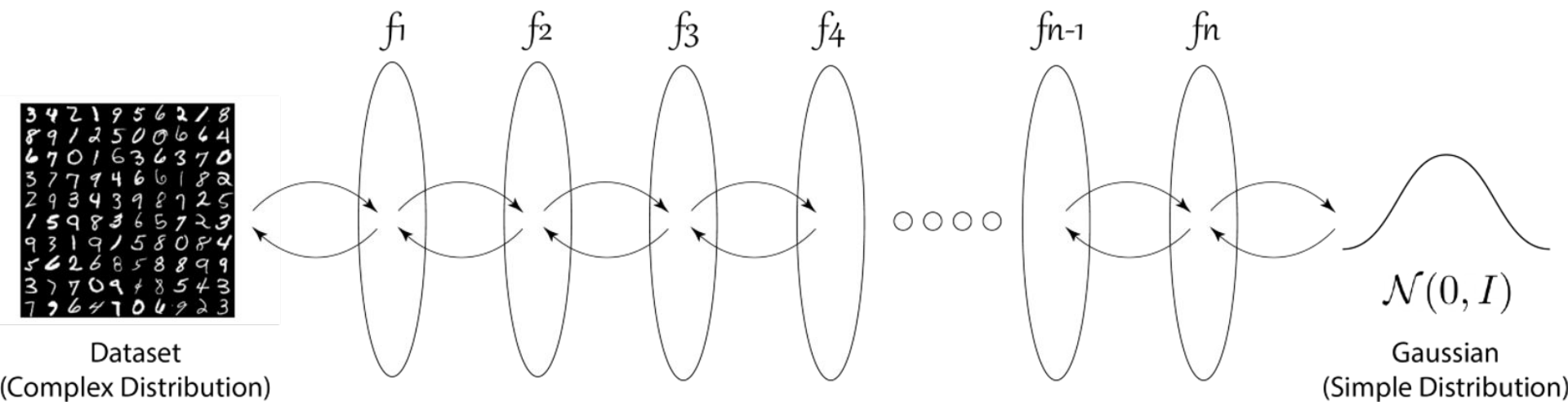
$$\frac{\partial \mathbf{f}(x)}{\partial x} = \begin{pmatrix} \mathbb{I}_d & 0 \\ \frac{\partial z_{1:d}}{\partial x_{1:d}} & \text{diag}(\exp(\mathbf{s}(x_{1:d}))) \end{pmatrix}$$

$$\left| \det \frac{\partial \mathbf{f}(x)}{\partial x} \right| = \exp\left(\sum_{j=d+1}^D \mathbf{s}(x_{1:d})_j \right)$$

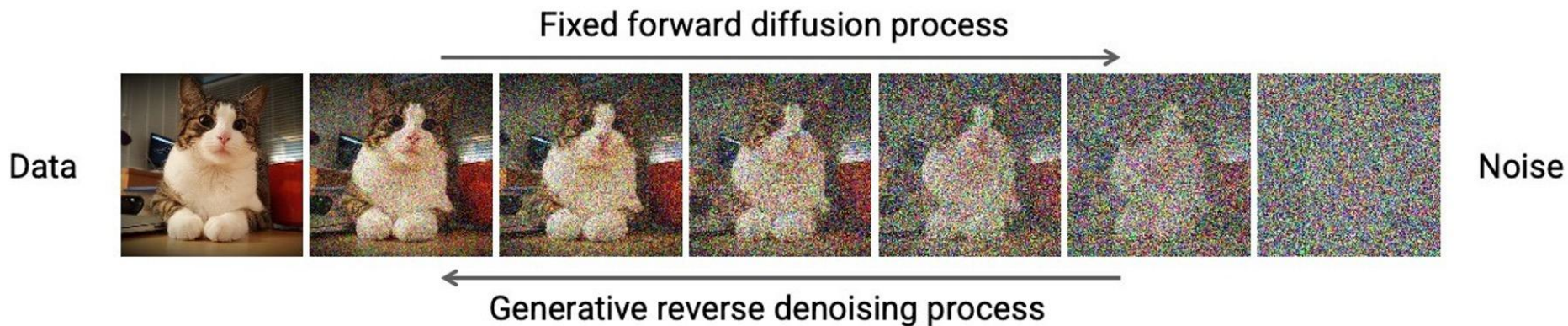
Results



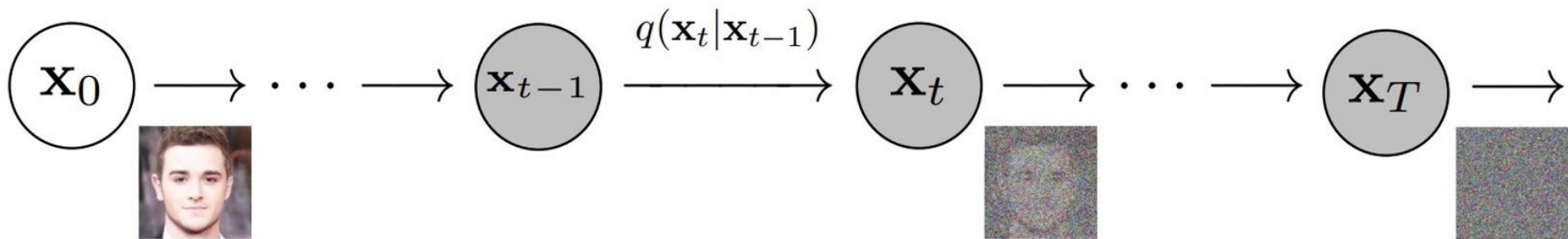
Normalizing flow in a nutshell



New model – Diffusion!

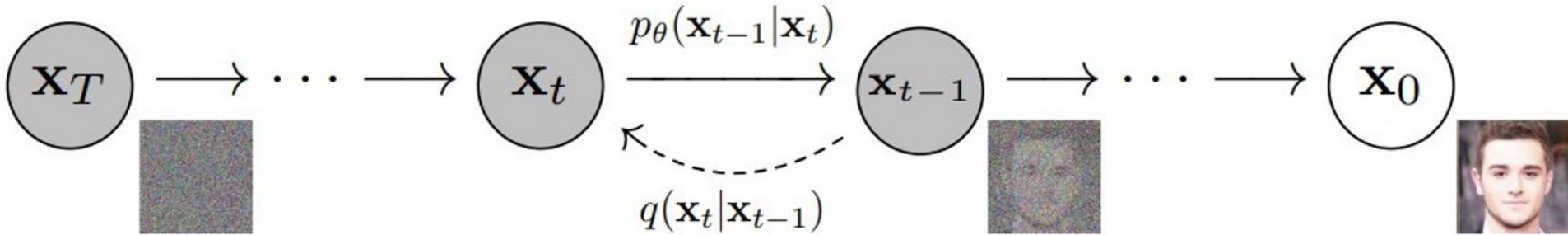


Salt and pepper to taste

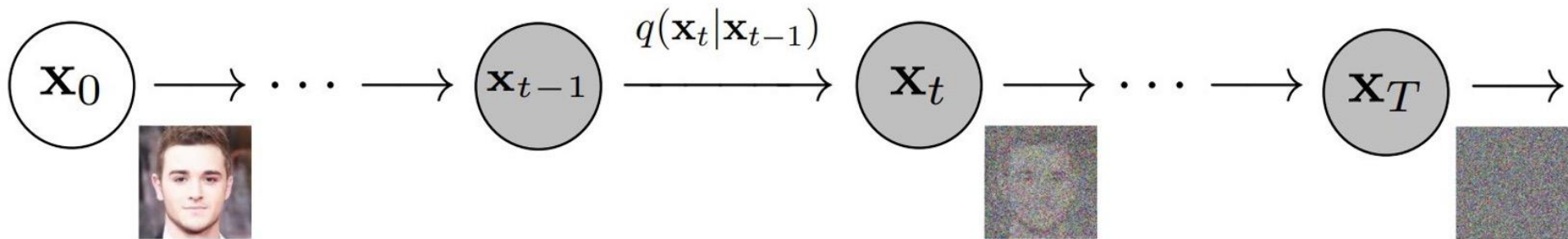


$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \boldsymbol{\mu}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \boldsymbol{\Sigma}_t = \beta_t \mathbf{I})$$

Remove salt and pepper



How does the image change?



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \boldsymbol{\mu}_t = \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \boldsymbol{\Sigma}_t = \beta_t\mathbf{I})$$

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}_t$$

$$\boldsymbol{\epsilon}_t \sim N(0, \mathbf{I}), \quad \alpha_t = 1 - \beta_t$$

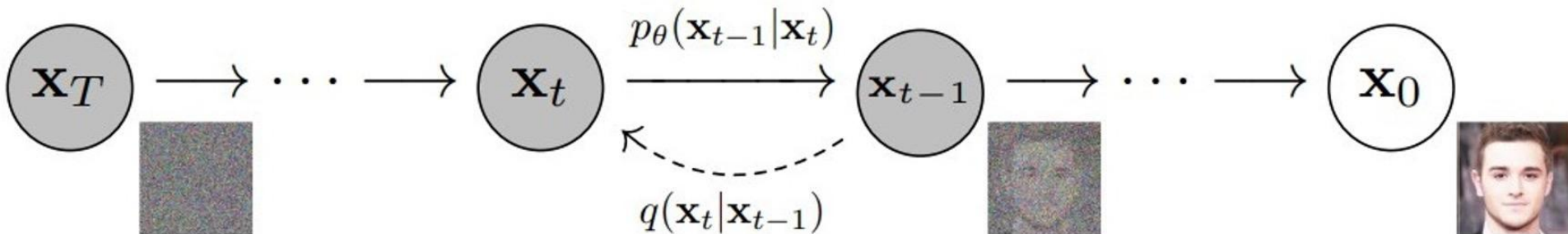
$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$$

$$\boldsymbol{\epsilon} \sim N(0, \mathbf{I}), \quad \bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\boldsymbol{\epsilon}\right) + \tilde{\beta}_t\mathbf{z}$$

$$\mathbf{z} \sim N(0, \mathbf{I}), \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

How to get the image back?



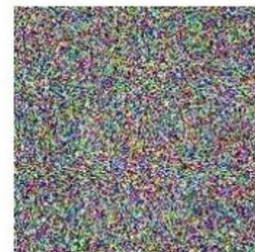
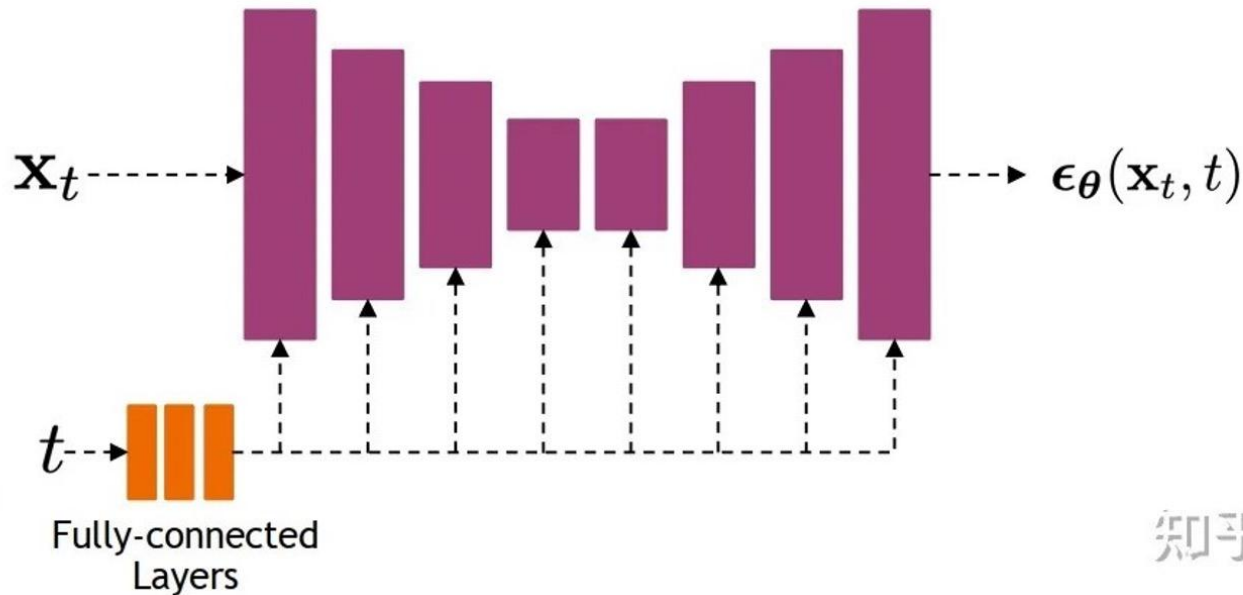
$$\hat{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z \quad z \sim N(0, I), \quad \sigma_t = \text{const}$$

$$L_t = \|x_{t-1} - \hat{x}_{t-1}\|_2^2 \propto \|\epsilon - \epsilon_{\theta}(x_t, t)\|_2^2 \rightarrow \min_{\theta}$$

We should probably pass the timestep



Time Representation



知乎 @华年SS

Training algorithm

Algorithm 1 Training

1: **repeat**

2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$

3: $t \sim \text{Uniform}(\{1, \dots, T\})$

4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

5: Take gradient descent step on

$$\nabla_{\theta} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$$

6: **until** converged

Generation

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

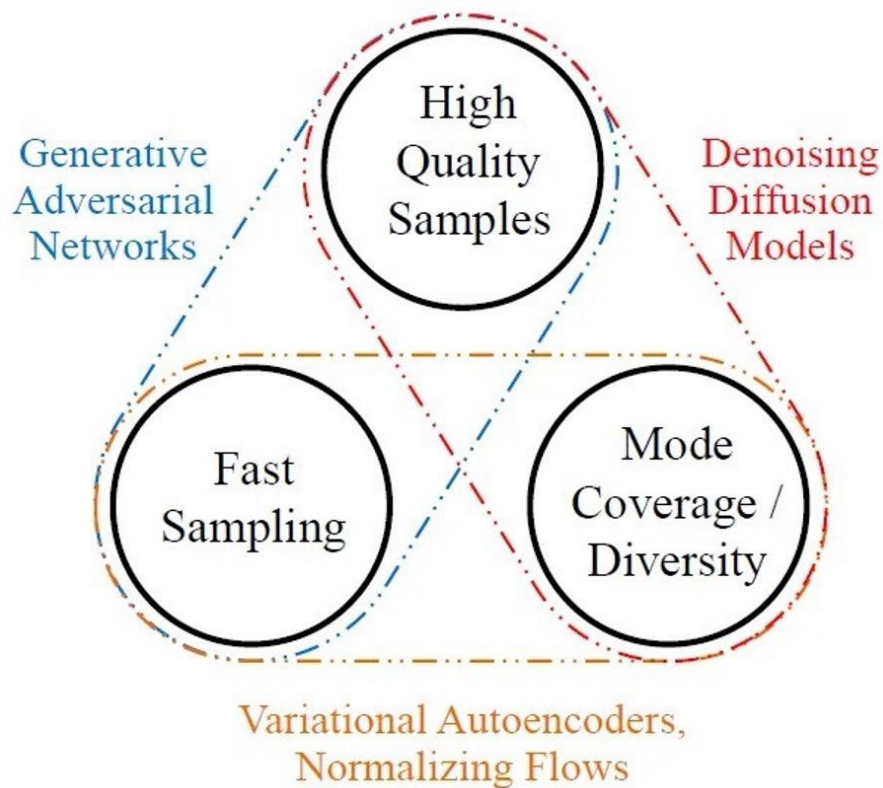

Scheduling

$$T = 1000$$

$$\beta_1 < \beta_2 < \dots < \beta_t < \dots < \beta_T$$

$$\beta_1 = 0.0001, \beta_T = 0.02$$

Trilemma



VAE Math

$$\mathbf{z} \sim p(\mathbf{z})$$

$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

$$\begin{aligned} p(\mathbf{x}) &= \int p(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) \, d\mathbf{z} \\ &= \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [p(\mathbf{x}|\mathbf{z})] \\ &\approx \frac{1}{K} \sum_k p(\mathbf{x}|\mathbf{z}_k) \end{aligned}$$

But $\mathbf{z} \in \mathbb{R}^M$

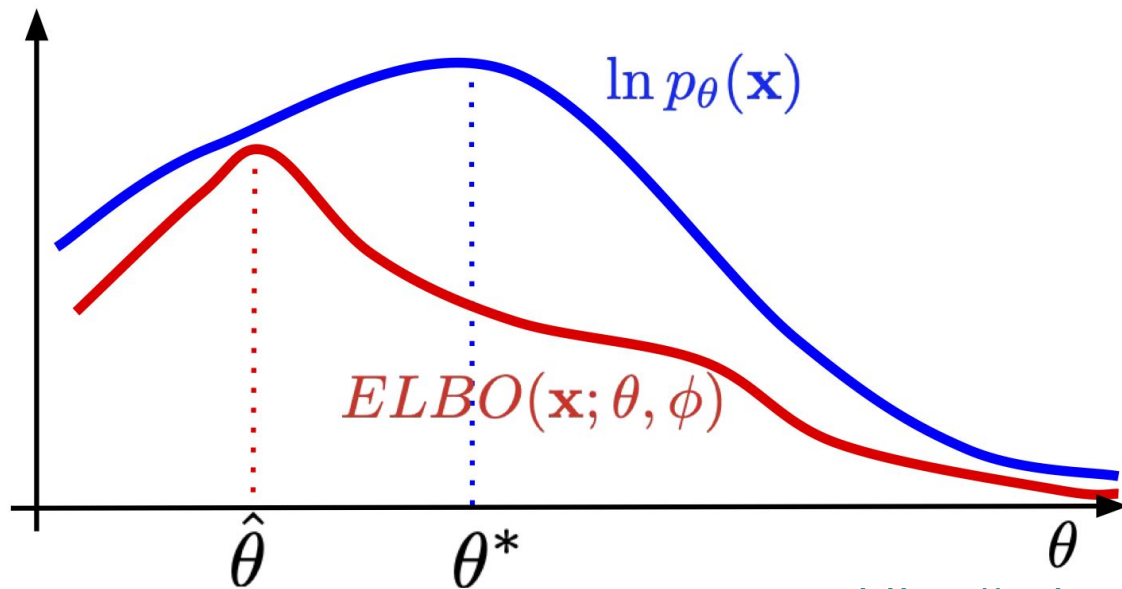
Oh no, we have a problem!

VAE

$$\begin{aligned}\ln p(\mathbf{x}) &= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p(\mathbf{x})] \\&= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\ln \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{z}|\mathbf{x})} \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\ln p(\mathbf{x}|\mathbf{z}) \frac{p(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\ln p(\mathbf{x}|\mathbf{z}) - \ln \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} + \ln \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p(\mathbf{x}|\mathbf{z})] - KL[q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})] + KL[q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] .\end{aligned}$$

VAE

$$\ln p(\mathbf{x}) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\ln p(\mathbf{x}|\mathbf{z})] - KL[q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})]}_{ELBO} + \underbrace{KL[q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}|\mathbf{x})]}_{\geq 0}$$



GAN Math

See <https://arxiv.org/pdf/1406.2661>